

Database Project – Step 3

Rawad ALLAM, Daïf MOINDJIE, Félix LANG, Seifeleslam ELGHABATY

1. Claude AI prompt:

In « step 3 - prompt.txt »

2. Views (in « step 3 - views.txt »)

- Hide sensitive artist contact info:

```
CREATE VIEW vw_artist_public AS
SELECT
  Aname,
  Acity,
  Abio,
  Awebsite,
  AsocialMedia,
  isActive
FROM Artists;
```

Objective: Hides private fields (Aemail, Aphone, AbirthYear, ArtistID) from public-facing queries. Only expose what a visitor of the platform needs to see.

- Member profile without personal details:

```
CREATE VIEW vw_member_public AS
SELECT
  MemberID,
  CMname,
  CMcity,
  MembershipType
FROM CommunityMember;
```

Objective: Hides sensitive data (CMemail, CMphone, CMbirthYear) from users who only need to reference a member by name or membership type (e.g., for booking lists).

- Available artworks with artist name:

```
CREATE VIEW vw_available_artworks AS
SELECT
  aw.ArtworkID,
  aw.AWtitle,
  aw.AWtype,
  aw.AWprice,
  aw.AWmedium,
  aw.AWdimensions,
  ar.Aname    AS ArtistName,
  ar.Acity    AS ArtistCity
FROM Artworks aw
JOIN Artists ar ON aw.ArtistID = ar.ArtistID
WHERE aw.AWstatus = 'Available';
```

Objective: Simplifies any query that needs to display artworks for sale — avoids repeating the JOIN and WHERE filter every time. Useful for a storefront or catalogue page.

2. Indexes (in « step 3 - indexes.txt »)

- Index on Artworks.AWstatus

```
CREATE INDEX idx_artworks_status ON Artworks(AWstatus);
```

Why? Filtering artworks by status ('Available', 'Sold', etc.) is a good choice of index. Without an index, every query scans the entire table. Since status has low cardinality, this index is lightweight but highly effective for frequent filters.

- Index on Booking.WorkshopID

```
CREATE INDEX idx_booking_workshopid ON Booking(WorkshopID);
```

Why? Without this index, every booking insertion triggers a full table scan.

- Index on Review.ArtworkID

```
CREATE INDEX idx_review_artworkid ON Review(ArtworkID);
```

Why? Since ArtworkID is a foreign key in Review but not the primary key, it won't be automatically indexed so this index must be added explicitly.

3. Triggers (in « step 3 - triggers.txt »)

- Checking workshop date (workshop booking date must not be after workshop date)

```
DELIMITER //
CREATE TRIGGER check_booking_date BEFORE INSERT ON Booking FOR EACH ROW
BEGIN
    DECLARE ws_date DATE;
    SELECT WSdate INTO ws_date FROM Workshops WHERE WorkshopID =
NEW.WorkshopID;
    IF NEW.BookingDate > ws_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking date cannot be after the workshop date.';
    END IF;
END;
//
DELIMITER ;
```

- Checking workshop capacity (we want to prevent booking a workshop over capacity)

```
DELIMITER //
CREATE TRIGGER trg_check_capacity BEFORE INSERT ON Booking FOR EACH ROW
BEGIN
    DECLARE current_count INT;
    DECLARE max_cap INT;

    SELECT COUNT(*) INTO current_count FROM Booking WHERE WorkshopID =
NEW.WorkshopID;

    SELECT MaxParticipants INTO max_cap FROM Workshops WHERE WorkshopID =
NEW.WorkshopID;

    IF current_count >= max_cap THEN
        SIGNAL SQLSTATE '45000'
```

```

        SET MESSAGE_TEXT = 'Workshop is fully booked. No seats available.';
    END IF;
END;
//
DELIMITER ;

```

- Logging changes (log any update to an exhibition)

-- Audit log table

```

CREATE TABLE ExhibitionAuditLog (
    LogID    INT AUTO_INCREMENT PRIMARY KEY,
    ExhibitionID INT,
    ChangedAt DATETIME DEFAULT NOW(),
    ChangedField VARCHAR(50),
    OldValue  VARCHAR(200),
    NewValue  VARCHAR(200)
);

```

DELIMITER //

```

CREATE TRIGGER trg_audit_exhibition_update
AFTER UPDATE ON Exhibitions
FOR EACH ROW
BEGIN
    IF OLD.EXtitle <> NEW.EXtitle THEN
        INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
        VALUES (OLD.ExhibitionID, 'EXtitle', OLD.EXtitle, NEW.EXtitle);
    END IF;
    IF OLD.EXstartDate <> NEW.EXstartDate THEN
        INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
        VALUES (OLD.ExhibitionID, 'EXstartDate', OLD.EXstartDate, NEW.EXstartDate);
    END IF;
    IF OLD.EXendDate <> NEW.EXendDate THEN
        INSERT INTO ExhibitionAuditLog (ExhibitionID, ChangedField, OldValue, NewValue)
        VALUES (OLD.ExhibitionID, 'EXendDate', OLD.EXendDate, NEW.EXendDate);
    END IF;
END;
//
DELIMITER ;

```

3. Stored procedures

- Create a Workshop and automatically register its instructor (artist)

DELIMITER //

```
CREATE PROCEDURE sp_create_workshop_and_register_artist(
    IN p_title    VARCHAR(100),
    IN p_date     DATE,
    IN p_price    DECIMAL(9,2),
    IN p_level    VARCHAR(12),
    IN p_duration INT,
    IN p_maxPart  INT,
    IN p_location VARCHAR(100),
    IN p_description VARCHAR(200),
    IN p_artistID INT
)
BEGIN
    DECLARE new_workshop_id INT;

    -- Create the workshop
    INSERT INTO Workshops (WStitle, WSdate, WSprice, WSlevel, DurationMinutes,
        MaxParticipants, WSlocation, WSdescription)
    VALUES (p_title, p_date, p_price, p_level, p_duration,
        p_maxPart, p_location, p_description);

    SET new_workshop_id = LAST_INSERT_ID();

    -- Automatically register the instructor (artist) as a participant
    INSERT INTO Participates_in (ArtistID, WorkshopID)
    VALUES (p_artistID, new_workshop_id);

    SELECT new_workshop_id AS CreatedWorkshopID;
END;
//
DELIMITER ;
```

- Function – Return the number of participants for a given workshop

DELIMITER //

```
CREATE FUNCTION fn_count_participants(p_workshopID INT)
```

```

RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE participant_count INT;
    SELECT COUNT(*) INTO participant_count
    FROM Booking
    WHERE WorkshopID = p_workshopID;
    RETURN participant_count;
END;
//
DELIMITER ;

```

4. Transactions

Scenario: Member purchases an artwork and books a related workshop

This transaction covers a realistic real-world flow: a member buys an artwork and simultaneously books a workshop taught by that artwork's artist. All steps must succeed together or the entire operation is rolled back.

The steps that must be atomic are:

Verify the artwork is available and the workshop has capacity

Mark the artwork as 'Sold'

Create a booking for the workshop

Create a review placeholder entry (optional but linked)

If anything fails → roll back everything

```
DELIMITER //
```

```

CREATE PROCEDURE sp_purchase_and_book(
    IN p_memberID INT,
    IN p_artworkID INT,
    IN p_workshopID INT,
    IN p_bookingDate DATE
)

```

```
BEGIN
```

```
-- ——— Declare variables —————
```

```
DECLARE v_status VARCHAR(9);
```

```

DECLARE v_currentCount INT;
DECLARE v_maxPart    INT;
DECLARE v_wsDate     DATE;
DECLARE v_artistID   INT;
DECLARE v_alreadyBooked INT;

-- Declare EXIT HANDLER: if anything goes wrong, rollback everything
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction failed. All changes have been rolled back.';
END;

-- ——— Start transaction —————
START TRANSACTION;

-- ——— Step 1: Check artwork is available —————
SELECT AWstatus, ArtistID
INTO v_status, v_artistID
FROM Artworks
WHERE ArtworkID = p_artworkID
FOR UPDATE; -- Lock the row to prevent concurrent purchases

IF v_status <> 'Available' THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Artwork is not available for purchase.';
END IF;

-- ——— Step 2: Check workshop capacity —————
SELECT WSdate, MaxParticipants
INTO v_wsDate, v_maxPart
FROM Workshops
WHERE WorkshopID = p_workshopID
FOR UPDATE; -- Lock to prevent race condition on capacity

SELECT COUNT(*)
INTO v_currentCount
FROM Booking
WHERE WorkshopID = p_workshopID;

```

```

IF v_currentCount >= v_maxPart THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Workshop is fully booked.';
END IF;

-- — Step 3: Check booking date is valid —————
IF p_bookingDate > v_wsDate THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Booking date cannot be after the workshop date.';
END IF;

-- — Step 4: Check member hasn't already booked this workshop ———
SELECT COUNT(*)
INTO v_alreadyBooked
FROM Booking
WHERE WorkshopID = p_workshopID
    AND MemberID = p_memberID;

IF v_alreadyBooked > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Member is already booked for this workshop.';
END IF;

-- — Step 5: Mark artwork as Sold —————
UPDATE Artworks
SET AWstatus = 'Sold'
WHERE ArtworkID = p_artworkID;

-- — Step 6: Create the workshop booking —————
INSERT INTO Booking (BookingDate, PaymentStatus, WorkshopID, MemberID)
VALUES (p_bookingDate, 'Pending', p_workshopID, p_memberID);

-- — Step 7: Verify the workshop's instructor is the artwork artist
--      and register them as participant if not already registered
IF NOT EXISTS (
    SELECT 1 FROM Participates_in
    WHERE ArtistID = v_artistID
        AND WorkshopID = p_workshopID
) THEN
    INSERT INTO Participates_in (ArtistID, WorkshopID)
    VALUES (v_artistID, p_workshopID);

```

```
END IF;
```

```
-- — All good: commit
```

```
COMMIT;
```

```
SELECT 'Transaction successful.' AS Result,
```

```
    p_artworkID      AS PurchasedArtwork,
```

```
    p_workshopID     AS BookedWorkshop,
```

```
    p_memberID       AS MemberID;
```

```
END;
```

```
//
```

```
DELIMITER ;
```

Usage:

```
CALL sp_purchase_and_book(
```

```
    7,      -- MemberID
```

```
    42,     -- ArtworkID
```

```
    3,      -- WorkshopID
```

```
    '2025-05-20' -- BookingDate
```

```
);
```